[Paper Title]

Anonymous Author(s)
Affiliation
Address
email

Abstract

1 Introduction

Contributions.

- TODO: contribution 1.
- TODO: contribution 2.
- TODO: contribution 3.

2 Related Work

TODO: subtopic 1.

TODO: subtopic 2.

TODO: subtopic 3.

3 Method

3.1 Preliminaries and Problem Setup

3.2 Model Architecture

Our policy is a memory-augmented vision-language-action (VLA) model built on top of the $\pi_{0.5}$ backbone. We keep the backbone weights frozen and inject the per-episode memory sequence assembled in Sec. 3.3 as additional context tokens.

$\pi_{0.5}$ **backbone.** At history timestep t, the policy receives language tokens $\boldsymbol{\ell} = \mathrm{Tok}_{\mathrm{text}}(l) \in \mathbb{R}^{N_\ell \times d}$ and multi-view image tokens $\mathbf{o}_t = \mathrm{Tok}_{\mathrm{img}}(I_t) \in \mathbb{R}^{N_o \times d}$ extracted by a frozen SigLIP2 encoder. $\pi_{0.5}$ follows the π_0 multi-expert design and is composed of (i) a *VLM expert* E_{vlm} that fuses language and vision and (ii) an *action expert* E_{act} that predicts an action chunk conditioned on the VLM features and a denoising timestep τ. Both experts are L-layer transformers connected by a *blockwise causal attention* operator

$$\mathbf{y} \;=\; \mathrm{BlockAttn}(\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_G), \tag{1}$$

in which attention is bidirectional within each block $\mathbf{x}_i$ and causal across blocks (block i may attend blocks $1{:}i$ but not future blocks). The action expert is conditioned on τ via AdaLN-Zero modulation:

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

with action latent $\mathbf{s}_t^k$ at layer k,

$$\tilde{\mathbf{s}}_t^k = \mathbf{s}_t^{k-1} + g_\tau \odot \mathrm{BlockAttn}^k\big(\mathrm{Norm}^k_{\mathrm{vlm}}(\mathbf{u}_t^{k-1}),\, \mathrm{Norm}^k_{\mathrm{act}}(\mathbf{s}_t^{k-1}, \tau)\big), \quad \mathbf{s}_t^k = \tilde{\mathbf{s}}_t^k + \tilde{g}_\tau \odot \mathrm{MLP}^k_{\mathrm{act}}\big(\mathrm{Norm}^k_{\mathrm{act}}(\tilde{\mathbf{s}}_t^k, \tau)\big), \tag{2}$$

where $g_\tau, \tilde{g}_\tau$ are gates conditioned on τ. Unlike the original $\pi_{0.5}$, which uses the VLM expert to also generate low-level language commands, we keep the VLM as a pure feature extractor; this leaves a clean interface at which memory can be injected.

Memory token block. We expose the memory sequence $\mathcal{M}$ produced in Sec. 3.3 to the backbone as a single token block $\mathbf{M}_t \in \mathbb{R}^{B \times d}$ with fixed budget B (zero-padded when fewer tokens are available). The two-token-kind structure of $\mathcal{M}$ — TokenDropped keyframe patches and MotionJEPA motion tokens — is opaque to the backbone: from the policy's perspective $\mathbf{M}_t$ is just a length-B sequence of d-dimensional vectors carrying time tags.

Three integration mechanisms. We systematically study three places at which $\mathbf{M}_t$ can enter the backbone, trading parameter count, attention budget, and isolation of the pretrained VLM features against each other.

(1) Memory-as-Context (MaC). The simplest variant: concatenate $\mathbf{M}_t$ with the standard inputs of the VLM expert,

$$\mathbf{u}_t^0 \;=\; [\,\mathbf{M}_t;\, \mathbf{o}_t;\, \boldsymbol{\ell}\,], \qquad \mathbf{u}_t \;=\; E_{\mathrm{vlm}}(\mathbf{u}_t^0). \tag{3}$$

MaC introduces *no* new parameters and reuses the existing blockwise causal attention (1); its cost is that memory tokens compete with raw observation tokens for the same attention budget inside E_{vlm}, and any noise in $\mathbf{M}_t$ propagates through every downstream feature.

(2) Memory-as-Modulator (MaM). Instead of feeding memory through the VLM, we use it to *modulate* the action expert. At each layer k, the action latent $\tilde{\mathbf{s}}_t^k$ first cross-attends $\mathbf{M}_t$, and the result is mapped to per-feature scale/shift parameters via a zero-initialized MLP that is then composed with AdaLN:

$$\mathbf{r}_t^k = \mathrm{Attn}^k_{\mathrm{mod}}\big(Q{=}\tilde{\mathbf{s}}_t^k,\; K{=}\mathbf{M}_t,\; V{=}\mathbf{M}_t\big), \tag{4}$$

$$(\gamma_t^k, \beta_t^k) = \mathrm{MLP}^k_{\mathrm{mod}}(\mathbf{r}_t^k), \tag{5}$$

$$\widehat{\mathbf{s}}_t^k = \gamma_t^k \odot \mathrm{Norm}^k_{\mathrm{mod}}(\tilde{\mathbf{s}}_t^k) + \beta_t^k, \tag{6}$$

$$\mathbf{s}_t^k = \widehat{\mathbf{s}}_t^k + \tilde{g}_\tau \odot \mathrm{MLP}^k_{\mathrm{act}}\big(\mathrm{Norm}^k_{\mathrm{act}}(\widehat{\mathbf{s}}_t^k, \tau)\big). \tag{7}$$

$\mathrm{MLP}^k_{\mathrm{mod}}$ is initialized so that $(\gamma_t^k, \beta_t^k) = (1, 0)$ at the start of fine-tuning, making the modulation an exact identity at step zero and protecting the pretrained $\pi_{0.5}$ behavior. MaM adds a moderate number of parameters ($\sim$80M, see Sec. 3.5) and, importantly, leaves the VLM expert untouched.

(3) Memory-as-Expert (MaE). We allocate a dedicated third expert E_{mem} that processes $\mathbf{M}_t$ through its own L layers of self-attention plus MLP:

$$\tilde{\mathbf{v}}_t^k = \mathbf{v}_t^{k-1} + \mathrm{BlockAttn}^k\big(\mathrm{Norm}^k_{\mathrm{mem}}(\mathbf{v}_t^{k-1})\big), \quad \mathbf{v}_t^k = \tilde{\mathbf{v}}_t^k + \mathrm{MLP}^k_{\mathrm{mem}}\big(\mathrm{Norm}^k_{\mathrm{mem}}(\tilde{\mathbf{v}}_t^k)\big), \tag{8}$$

with $\mathbf{v}_t^0 = \mathbf{M}_t$. The action expert's update (2) is then extended to attend *both* VLM and memory features as additional context blocks:

$$\tilde{\mathbf{s}}_t^k = \mathbf{s}_t^{k-1} + g_\tau \odot \mathrm{BlockAttn}^k\big(\mathrm{Norm}^k_{\mathrm{mem}}(\mathbf{v}_t^{k-1}),\, \mathrm{Norm}^k_{\mathrm{vlm}}(\mathbf{u}_t^{k-1}),\, \mathrm{Norm}^k_{\mathrm{act}}(\mathbf{s}_t^{k-1}, \tau)\big). \tag{9}$$

The blockwise causal mask is configured asymmetrically: the VLM expert and memory expert each use self-attention only and *do not* attend each other; the action expert is the sole consumer that attends both. This asymmetry preserves the pretrained $\pi_{0.5}$ representations exactly while allocating a separate transformer's worth of capacity ($\sim$190M parameters) to memory processing.

3.3 Memory Token Generation and Training

The memory passed to the VLA is a sparse, time-aligned sequence of two complementary token kinds drawn from a single robot video of length T: (i) *keyframe tokens*, which anchor the scene at semantic transitions, and (ii) *motion tokens*, fixed-stride summaries of short-range visual change produced by a JEPA-style encoder. Both kinds are computed offline from the raw video and a synchronized proprioceptive trace; no gradient flows back into the perception backbones during VLA training.

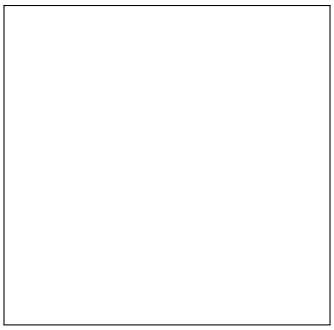

Figure 1: TODO: method overview figure. Left: offline memory pipeline (CLIP keyframe detection $\rightarrow$ TokenDrop $\rightarrow$ MotionJEPA motion tokens) producing $\mathbf{M}_t$. Right: the three injection sites in the $\pi_{0.5}$ backbone — Memory-as-Context into the VLM input, Memory-as-Modulator into AdaLN of the action expert, and Memory-as-Expert as a dedicated third expert.

Notation. Let $\{I_t\}_{t=0}^{T-1}$ be the input frames and $\{s_t\}$ the proprioceptive state. We use a frozen vision tokenizer $\Phi_{\text{SigLIP}} : \mathbb{R}^{3\times H\times W} \rightarrow \mathbb{R}^{S\times E}$ with $S = 256$ patches and $E = 1152$. A separate frozen image–text backbone Φ_{CLIP} is used only as a scalar similarity function for keyframe detection. We write $x_t = \Phi_{\text{SigLIP}}(I_t) \in \mathbb{R}^{S\times E}$.

Keyframe detection. Keyframes mark moments when the scene changes meaningfully (object pickup, contact events, viewpoint shifts). We score each frame by the smoothed discrete derivative of CLIP image embeddings,

$$d_t \;=\; \left\|\Phi_{\text{CLIP}}(I_t) - \Phi_{\text{CLIP}}(I_{t-1})\right\|_2, \qquad \tilde{d}_t \;=\; (k_{\text{rbf}} * d)_t, \tag{10}$$

where k_{rbf} is a small RBF kernel that suppresses frame-rate jitter. Keyframes are local maxima of $\tilde{d}_t$ exceeding a threshold τ, with the first frame $t = 0$ always retained as an anchor. The detector is purely deterministic preprocessing and has no learnable parameters; τ is a single hyperparameter chosen so that the average inter-keyframe gap is ~ 32 frames.

Keyframe TokenDrop. Storing the full S-token grid for every keyframe is wasteful: most of the image (background, robot base, table) repeats across keyframes. For every keyframe k_i with $i > 0$ we retain only the patches that actually changed relative to its *previous* keyframe k_{i-1}. Concretely, on the 16×16 patch grid we mark a patch as *significant* if its mean per-pixel absolute difference between I_{k_i} and $I_{k_{i-1}}$ exceeds δ, and we keep the union of all significant patches together with the patches lying inside their axis-aligned bounding rectangle (the latter preserves spatial contiguity for downstream attention). The first keyframe k_0 is always kept in full (S tokens); each subsequent keyframe contributes $|\mathcal{P}_i| \leq S$ tokens. TokenDrop is also parameter-free.

Motion tokens (MotionJEPA encoding). Independently of keyframe placement, we tile the video with non-overlapping windows of fixed stride Δ (we use $\Delta = 8$). For each window $[t, t+\Delta]$ a trainable *MotionEncoder* f_θ compresses the visual change into M latent tokens $m_t \in \mathbb{R}^{M\times D_m}$:

$$m_t \;=\; f_\theta(\, x_t,\; x_{t+\Delta},\; \Delta s_t\,), \qquad \Delta s_t = s_{t+\Delta} - s_t. \tag{11}$$

Internally, f_θ (i) computes the patchwise visual delta $\Delta x = x_{t+\Delta} - x_t \in \mathbb{R}^{S\times E}$, (ii) spatially average-pools Δx to a single global delta token in $\mathbb{R}^{1\times E}$ and projects it to the encoder width H, (iii) projects Δs_t to H and adds it to the pooled delta token (this is the only place proprioception enters the memory pipeline), (iv) appends M learned read-out queries at the tail, and (v) runs a small causal transformer (RMSNorm, 1D-RoPE causal self-attention, SwiGLU FFN) so that each query attends to the fused delta token before it. The last M positions are projected to motion dimension D_m and constitute m_t.

Memory sequence assembly. The final memory for an episode is the time-ordered concatenation

$$\mathcal{M} \;=\; \left[\, x_{k_0}^{\text{full}},\; m_0,\; m_\Delta,\; \ldots,\; x_{k_i}^{\text{TD}},\; \ldots \,\right], \tag{12}$$

where each entry carries its source timestamp as a positional tag. Compared to the naive grid of $T \cdot S$ patch tokens this yields a roughly order-of-magnitude reduction in token count while preserving both long-range scene anchors and short-range dynamics (concrete ratios are reported in Sec. 3.5).

Training objective. Keyframe selection and TokenDrop are training-free, so the only learnable parameters in the memory pipeline are those of the MotionEncoder f_θ. We train f_θ jointly with an auxiliary *MotionDiT* decoder g_ϕ under a JEPA-style future-prediction objective on the SigLIP token space. Given $(x_t, x_{t+\Delta})$, we sample a diffusion-style timestep $k \sim \mathcal{U}\{1, \ldots, K\}$ and ask g_ϕ to predict the (token-normalized) future tokens from the current tokens, the motion code, and k:

$$\hat{x}^{(k)}_{t+\Delta} \;=\; g_\phi(\, x_t,\; m_t,\; k\,), \qquad m_t = f_\theta(x_t, x_{t+\Delta}, \Delta s_t), \tag{13}$$

$$\mathcal{L}_{\text{pred}} \;=\; \mathbb{E}_{t,\,k}\Big[\, \big\| \hat{x}^{(k)}_{t+\Delta} \;-\; \text{TokenNorm}(x_{t+\Delta}) \big\|_2^2 \,\Big]. \tag{14}$$

TokenNorm is a per-feature standardization applied to the target SigLIP tokens to stabilize the regression target. g_ϕ is a small DiT with AdaLN-zero timestep modulation and a tied gated input/output projection initialized to zero, so at $k = 0$ the decoder acts as a normalized identity copy of x_t and only learns residual corrections driven by m_t. This zero-initialized residual head is the sole reason the loss is stable when f_θ is randomly initialized: gradients flow through g_ϕ into f_θ only via the correction term, forcing m_t to encode exactly the information needed to bridge $x_t \to x_{t+\Delta}$.

Crucially, g_ϕ exists only during pre-training. At deployment the robot has already executed both endpoints of every motion window, so memory tokens are produced by a single forward pass through Φ_{SigLIP} and f_θ; no decoding is performed and g_ϕ is discarded. The trained f_θ, together with the parameter-free keyframe and TokenDrop modules, is what ultimately writes into $\mathcal{M}$.

3.4 Training Objective

The full system has two distinct loss surfaces, optimized in two stages: the JEPA-style future-prediction loss $\mathcal{L}_{\text{pred}}$ that trains the MotionEncoder f_θ (already given in Sec. 3.3), and a flow-matching loss $\mathcal{L}_{\text{FM}}$ that trains the VLA backbone together with the integration module.

Flow-matching action loss. Following $\pi_{0.5}$, the policy predicts an H-step action chunk $\mathbf{A}_t = [a_t, a_{t+1}, \ldots, a_{t+H-1}] \in \mathbb{R}^{H \times D}$ via conditional flow matching. Given a ground-truth chunk $\mathbf{A}_t^{\text{data}} \sim p_{\text{data}}$, Gaussian noise $\mathbf{A}_t^{\text{noise}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and a denoising timestep $\tau \sim \mathcal{U}(0, 1)$, we form the linear interpolant

$$\mathbf{A}_t^\tau \;=\; (1-\tau)\,\mathbf{A}_t^{\text{data}} \;+\; \tau\,\mathbf{A}_t^{\text{noise}}, \tag{15}$$

whose constant time-derivative is $\frac{d}{d\tau}\mathbf{A}_t^\tau \;=\; \mathbf{A}_t^{\text{noise}} - \mathbf{A}_t^{\text{data}}$. The action expert parameterizes a conditional velocity field $\mathbf{v}_\theta(\mathbf{A}_t^\tau, \tau, \mathbf{h}_t)$, where the context features $\mathbf{h}_t$ are the layer-L outputs of the integrated backbone — so $\mathbf{h}_t$ depends on the chosen integration mechanism: in MaC the memory enters $\mathbf{h}_t$ through the VLM expert (3); in MaM through the per-layer modulation (4–7); in MaE through the cross-expert attention (9). The training objective is

$$\mathcal{L}_{\text{FM}} \;=\; \mathbb{E}_{t,\,\tau,\,\mathbf{A}_t^{\text{data}},\,\mathbf{A}_t^{\text{noise}}}\Big[\, \big\| \mathbf{v}_\theta\big(\mathbf{A}_t^\tau, \tau, \mathbf{h}_t\big) \;-\; \big(\mathbf{A}_t^{\text{noise}} - \mathbf{A}_t^{\text{data}}\big) \big\|_2^2 \,\Big]. \tag{16}$$

At inference the action chunk is generated by integrating $\mathbf{v}_\theta$ from $\tau = 1$ (pure noise) to $\tau = 0$ (data) through a small number of Euler steps.

Two-stage training. The two losses are optimized sequentially, not jointly, because the memory pipeline encodes a task-agnostic visual-dynamics prior whereas the integration module is task-specific.

Stage 1: Memory-pipeline pre-training. On unlabeled robot videos we jointly train the MotionEncoder f_θ and the auxiliary MotionDiT decoder g_ϕ to minimize $\mathcal{L}_{\text{pred}}$ (Sec. 3.3). The SigLIP and CLIP backbones, the keyframe detector, and the TokenDrop module are training-free throughout. After Stage 1, g_ϕ is discarded and f_θ is frozen.

Stage 2: VLA fine-tuning. For every demonstration episode we *pre-compute and cache* the memory sequence $\mathcal{M} \to \mathbf{M}_t$ using the frozen pipeline of Stage 1. SigLIP2 visual tokens are likewise pre-computed and cached, following the $\pi_{0.5}$ recipe. Trainable parameters in Stage 2 are exactly: (i) the VLM expert, (ii) the action expert, and (iii) the integration module — which contributes 0 new parameters for MaC (Eq. 3), $\sim$80M for MaM (Eqs. 4–7), or $\sim$190M for MaE (Eqs. 8–9). The single optimization objective is $\mathcal{L}_{\text{FM}}$. No proprioceptive state enters the backbone in Stage 2 (proprioception only participates in Sec. 3.3 when forming motion tokens).

This decoupling is what makes the same memory pipeline reusable across all three integration variants and across downstream tasks: Stage 1 is run once; Stage 2 is repeated for each (task, integration) configuration.

Table 1: TODO: main results. Brief description plus the key take-away message.

Method	Metric A	Metric B
Baseline	TODO	TODO
Ours	TODO	TODO

3.5 Implementation Details

Memory pipeline. SigLIP2 provides $S = 256$ patch tokens of width $E = 1152$ per 384×384 frame. CLIP keyframe detection uses an RBF smoother of width 3 frames and a threshold τ tuned so that the average inter-keyframe gap is ~ 32 frames. TokenDrop operates on the 16×16 patch grid; a patch is marked significant when its mean per-pixel absolute RGB difference w.r.t. the previous keyframe exceeds $\delta = 10^{-2}$, after which we add the patches inside the axis-aligned bounding rectangle of the significant set. The first keyframe is always retained in full at S tokens. The MotionJEPA encoder uses stride $\Delta = 8$, $M = 8$ readout queries, an encoder-width $H = 1024$, and a 4-layer causal transformer (RMSNorm, 1D-RoPE, SwiGLU). MotionDiT g_ϕ is a 6-layer DiT with AdaLN-zero modulation and a tied gated input/output projection initialized to zero. All memory-pipeline parameters total ~ 60M; SigLIP2 and CLIP are frozen.

Memory budget and compression. We cap $\mathbf{M}_t$ at $B = 512$ tokens of width $d = 1024$ for all integration variants, padding with zeros when the assembled $\mathcal{M}$ is shorter. Compared with the naive flattened representation of $T \cdot S$ patch tokens per episode, this is a $\sim 250\times$ reduction at $T \approx 500$, while preserving both the long-range keyframe anchors (the first keyframe contributes a full $S = 256$ tokens) and short-range dynamics ($M = 8$ motion tokens per $\Delta = 8$ frames).

Backbone and training schedule. The base $\pi_{0.5}$ policy processes 576 context tokens (512 visual $+ 64$ language) and contains 3.2B parameters. Stage 2 fine-tuning uses AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0), constant learning rate 5×10^{-5} with 5% warmup, gradient-norm clip 1.0, EMA 0.999, action horizon $H = 20$, and 80k steps. Effective batch size is 64 for MaC and MaM and 8 for MaE (the larger memory expert is the bottleneck). All runs use $4\times$NVIDIA A40 GPUs with FSDP, $\sim$3–4 days per configuration. Stage 1 uses the same optimizer with a shorter 40k-step schedule on a separate unlabeled video corpus. The SigLIP2 ViT is frozen and its outputs are cached on disk to amortize cost across configurations.

4 Experiments

4.1 Experimental Setup

4.2 Main Results

4.3 Ablation Studies

4.4 Analysis

5 Conclusion

References

References follow the acknowledgments in the camera-ready paper. Use unnumbered first-level heading for the references. Any choice of citation style is acceptable as long as you are consistent. It is permissible to reduce the font size to `small` (9 point) when listing the references. Note that the Reference section does not count towards the page limit.

[1] Alexander, J.A. & Mozer, M.C. (1995) Template-based algorithms for connectionist rule extraction. In G. Tesauro, D.S. Touretzky and T.K. Leen (eds.), *Advances in Neural Information Processing Systems 7*, pp. 609–616. Cambridge, MA: MIT Press.

[2] Bower, J.M. & Beeman, D. (1995) *The Book of GENESIS: Exploring Realistic Neural Models with the GEneral NEural SImulation System.* New York: TELOS/Springer–Verlag.

[3] Hasselmo, M.E., Schnell, E. & Barkai, E. (1995) Dynamics of learning and recall at excitatory recurrent synapses and cholinergic modulation in rat hippocampal region CA3. *Journal of Neuroscience* **15**(7):5249-5262.

A Technical appendices and supplementary material

Technical appendices with additional results, figures, graphs, and proofs may be submitted with the paper submission before the full submission deadline (see above). You can upload a ZIP file for videos or code, but do not upload a separate PDF file for the appendix. There is no page limit for the technical appendices.

Note: Think of the appendix as "optional reading" for reviewers. The paper must be able to stand alone without the appendix; for example, adding critical experiments that support the main claims to an appendix is inappropriate.